

Networking & Security Fundamentals

Complete Study Notes: Basic to Pro Level

TCP/IP | DNS | SSH | SSL/TLS

Includes diagrams, real-world examples, and exam-style practice questions with answers

Table of Contents

1. TCP/IP — Fundamentals of network communication
2. DNS — How domain names become IP addresses
3. SSH — Secure remote access and tunneling
4. SSL/TLS — Encryption and trust on the web

Chapter 1: TCP/IP

1.1 What Is Networking?

A computer network is a collection of devices connected together so they can exchange data. Every conversation between two devices on a network -- loading a webpage, sending an email, streaming a video -- happens by breaking data into small chunks called packets and sending them according to a shared set of rules called a protocol. TCP/IP (Transmission Control Protocol / Internet Protocol) is the foundational protocol suite that powers the modern Internet.

1.2 The OSI Model vs. the TCP/IP Model

The OSI (Open Systems Interconnection) model is a theoretical 7-layer framework used to describe how network communication works. The TCP/IP model is the practical 4-layer model that the real Internet is built on. Every OSI layer maps to a layer in the TCP/IP model, as shown below.

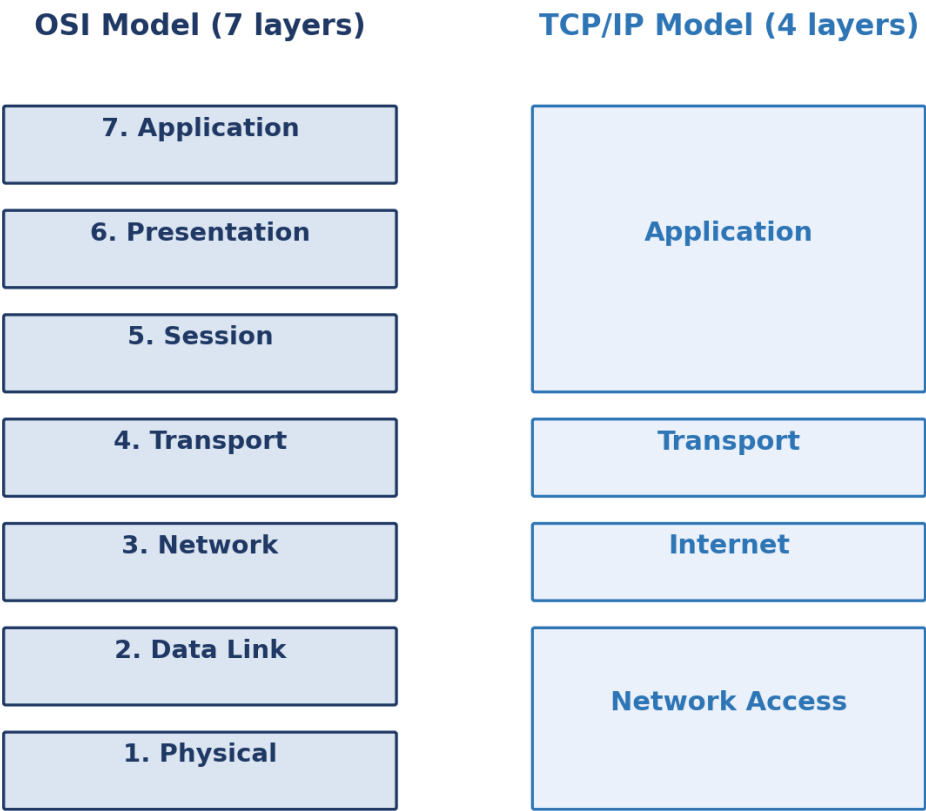


Figure 1.1 — OSI 7-layer model mapped to the TCP/IP 4-layer model

Quick layer reference

OSI Layer	TCP/IP Layer	Examples / Protocols	Data Unit
7. Application	Application	HTTP, HTTPS, FTP, SSH, DNS, SMTP	Data
6. Presentation	Application	SSL/TLS, encryption, encoding	Data
5. Session	Application	Session management, NetBIOS	Data

OSI Layer	TCP/IP Layer	Examples / Protocols	Data Unit
4. Transport	Transport	TCP, UDP	Segment (TCP) / Datagram (UDP)
3. Network	Internet	IP, ICMP, routers	Packet
2. Data Link	Network Access	Ethernet, Wi-Fi, MAC address, switches	Frame
1. Physical	Network Access	Cables, radio waves, NICs, hubs	Bits

1.3 IP Addressing

Every device on a TCP/IP network needs a unique address to be reachable: an IP address.

- **IPv4:** 32-bit address written as four decimal numbers (0-255) separated by dots, e.g. 192.168.1.10. Provides ~4.3 billion addresses -- now largely exhausted, which is why NAT and IPv6 exist.
- **IPv6:** 128-bit address written in hexadecimal, e.g. 2001:0db8:85a3::8a2e:0370:7334. Designed to solve IPv4 exhaustion with a virtually unlimited address space.
- **Public vs Private IP:** Private ranges (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16) are used inside local networks and are not routable on the public Internet; NAT (Network Address Translation) maps them to a public IP.
- **Subnetting & CIDR:** A subnet mask (e.g. 255.255.255.0, written as /24 in CIDR notation) splits an IP address into a network portion and a host portion, allowing large networks to be divided into smaller, manageable segments.

Worked example: CIDR

192.168.1.0/24 means the first 24 bits are the network portion, leaving 8 bits (256 addresses, 254 usable) for hosts -- from 192.168.1.1 to 192.168.1.254, with .0 as the network address and .255 as the broadcast address.

1.4 TCP vs UDP

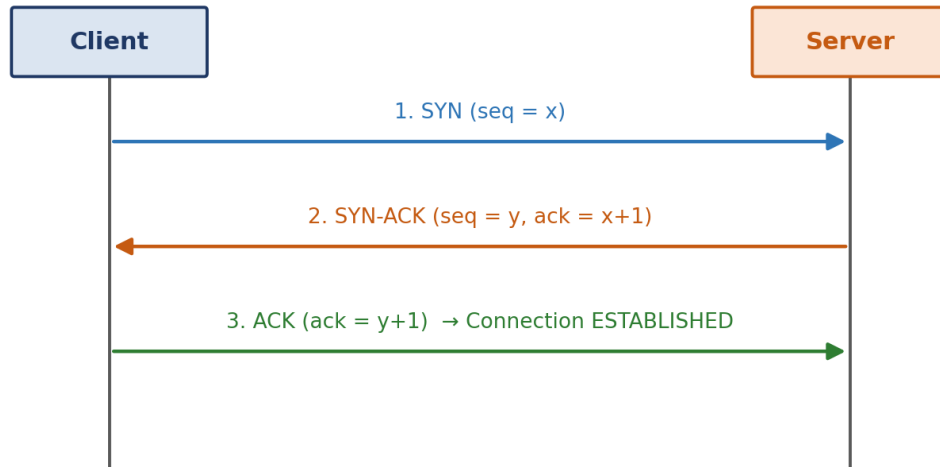
TCP and UDP are the two core Transport-layer protocols. They solve the same basic problem -- getting data from one application to another -- with very different trade-offs.

Feature	TCP	UDP
Connection	Connection-oriented (handshake required)	Connectionless
Reliability	Guaranteed delivery, retransmits lost packets	No delivery guarantee
Ordering	Packets arrive in order	No ordering guarantee
Speed	Slower (overhead of acknowledgment)	Faster, minimal overhead
Use cases	Web (HTTP/S), email, file transfer, SSH	DNS, video/voice streaming, online gaming

1.5 TCP Connection Setup: The 3-Way Handshake

Before any data flows over TCP, the client and server perform a 3-way handshake to synchronize sequence numbers and confirm both sides are ready to communicate.

TCP 3-Way Handshake (Connection Setup)



After the handshake, both sides can exchange data reliably over the TCP connection.

Figure 1.2 — TCP 3-way handshake (SYN, SYN-ACK, ACK)

- **SYN:** Client sends a segment with the SYN flag set and an initial sequence number (ISN).
- **SYN-ACK:** Server replies with its own SYN plus an ACK acknowledging the client's ISN + 1.
- **ACK:** Client acknowledges the server's ISN + 1. The connection is now ESTABLISHED.

The TCP segment header (simplified)

Simplified TCP Segment Header

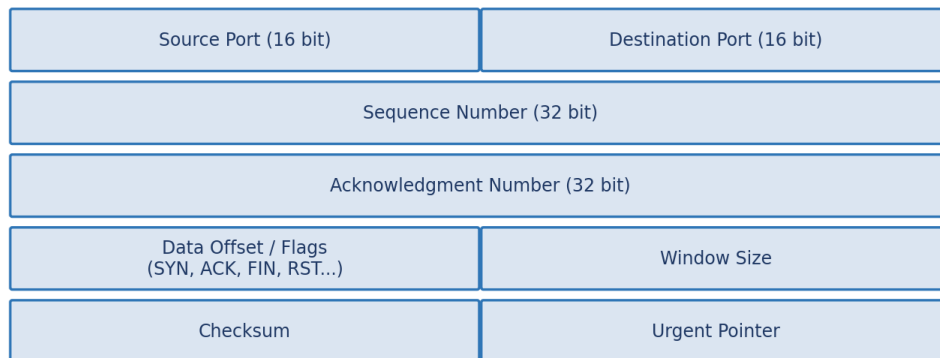


Figure 1.3 — Key fields inside a TCP segment

1.6 Connection Teardown: The 4-Way Termination

Closing a TCP connection normally uses four steps because each side must independently signal it has finished sending data (though ACK and FIN are sometimes combined).

TCP 4-Way Termination (Connection Teardown)

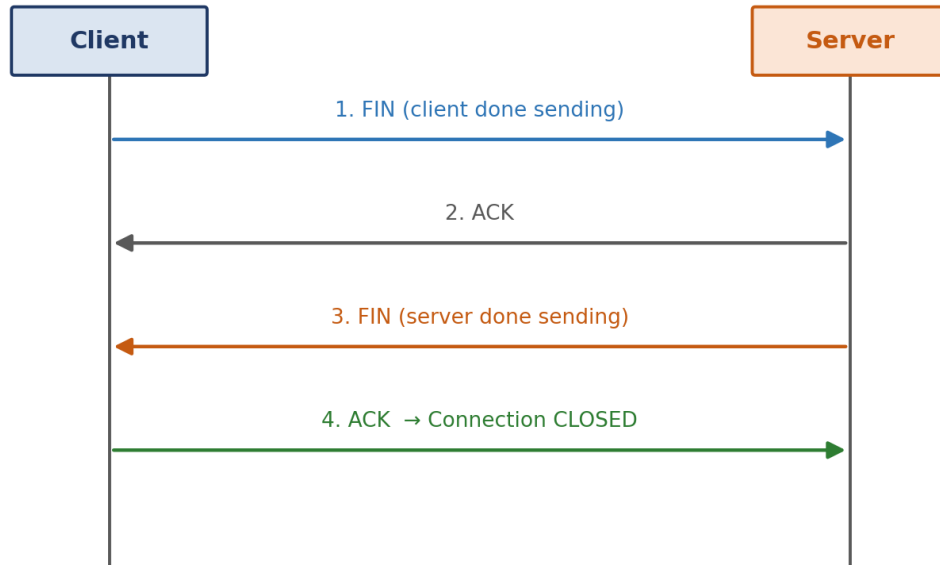


Figure 1.4 — TCP 4-way connection termination

1.7 Ports and Sockets

An IP address identifies a device; a port number identifies a specific application or service on that device. The combination of IP address + port + protocol is called a socket.

Port	Protocol	Service
20 / 21	TCP	FTP (data / control)
22	TCP	SSH
23	TCP	Telnet (insecure, legacy)
25	TCP	SMTP (email sending)
53	TCP/UDP	DNS
80	TCP	HTTP
443	TCP	HTTPS (TLS)
3306	TCP	MySQL

Ports 0-1023 are "well-known ports" reserved for standard services. Ports 1024-49151 are registered ports, and 49152-65535 are dynamic/private ports typically used as ephemeral client-side ports.

1.8 Advanced Concepts (Pro Level)

Flow control

TCP uses a sliding window mechanism so a fast sender does not overwhelm a slow receiver. The receiver advertises a "window size" telling the sender how much unacknowledged data it can have in flight.

Congestion control

TCP also protects the network itself from overload. Classic algorithms include:

- **Slow start:** The congestion window grows exponentially at the start of a connection until it hits a threshold.

- **Congestion avoidance:** After the threshold, growth becomes linear to probe for available bandwidth carefully.
- **Fast retransmit / fast recovery:** Duplicate ACKs signal a lost packet, so TCP retransmits without waiting for a full timeout.

TCP connection states

A TCP connection moves through a well-defined state machine: CLOSED → LISTEN → SYN_RCVD / SYN_SENT → ESTABLISHED → FIN_WAIT_1 → FIN_WAIT_2 → TIME_WAIT → CLOSED. The TIME_WAIT state exists to make sure any delayed packets from the old connection do not corrupt a new one.

NAT (Network Address Translation)

NAT allows many devices on a private network to share a single public IP address by rewriting source IP/port information on outbound packets and reversing it on the way back. This is why most home and office networks can run thousands of devices on one ISP-assigned IP.

Routing basics

Routers forward packets between networks based on destination IP addresses and a routing table. Routing protocols such as OSPF and BGP allow routers to automatically learn and share the best paths across large networks -- BGP, in particular, is the protocol that holds the global Internet together, routing traffic between Autonomous Systems (ASes).

MTU and fragmentation

The Maximum Transmission Unit (MTU) is the largest packet size a network link can carry (commonly 1500 bytes for Ethernet). If a packet exceeds the MTU of a link, it may be fragmented -- or dropped with an ICMP "Fragmentation Needed" message if the Don't Fragment flag is set, which is how Path MTU Discovery works.

1.9 Exam Practice: TCP/IP

Q1. Which layer of the OSI model is responsible for routing packets between networks?

- a) Data Link
- b) Network
- c) Transport
- d) Session

Answer: b) Network

Q2. Which protocol guarantees reliable, ordered delivery of data?

- a) UDP
- b) IP
- c) TCP
- d) ICMP

Answer: c) TCP

Q3. In the TCP 3-way handshake, what is the correct order of flags exchanged?

- a) ACK, SYN, SYN-ACK
- b) SYN, SYN-ACK, ACK
- c) SYN, ACK, FIN
- d) FIN, ACK, SYN

Answer: b) SYN, SYN-ACK, ACK

Q4. Which port does HTTPS use by default?

- a) 80
- b) 21
- c) 443
- d) 22

Answer: c) 443

Q5. What does CIDR notation /24 mean in an IPv4 address?

- a) 24 host bits
- b) 24 network bits
- c) 24 total addresses
- d) 24 subnets

Answer: b) 24 network bits (leaving 8 host bits)

Q6. Explain the difference between flow control and congestion control in TCP.

Answer: Flow control prevents a sender from overwhelming the receiver (based on the receiver's advertised window size). Congestion control prevents the sender from overwhelming the network itself (based on algorithms like slow start and congestion avoidance).

Q7. Why does a TCP connection enter the TIME_WAIT state before fully closing?

Answer: To ensure any delayed or duplicate packets from the closed connection are discarded by the network before a new connection reusing the same socket pair is established, preventing data corruption.

Q8. What problem does NAT solve, and what is one downside of relying on it?

Answer: NAT lets many private devices share one public IP address, conserving scarce IPv4 addresses. A downside is that it complicates hosting servers or protocols that embed IP addresses in their payload, and can break certain peer-to-peer connections without additional techniques like STUN/TURN.

Chapter 2: DNS (Domain Name System)

2.1 What Is DNS and Why Does It Exist?

Computers address each other using IP addresses (e.g. 93.184.216.34), but humans remember names (e.g. example.com) far more easily. DNS is the distributed, hierarchical system that translates human-readable domain names into IP addresses -- often called "the phonebook of the Internet."

2.2 The DNS Resolution Process

When you type a URL into a browser, several servers cooperate to resolve the name to an IP address:

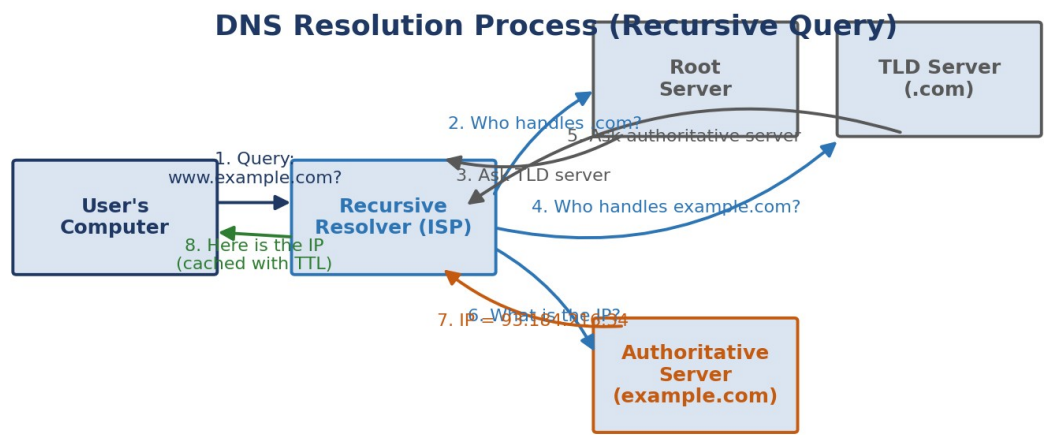


Figure 2.1 — Recursive DNS resolution from client to authoritative server

- **Recursive resolver:** Usually run by your ISP or a public service (e.g. 8.8.8.8, 1.1.1.1). It does the legwork of querying other servers on your behalf.
- **Root server:** Knows which servers are responsible for each top-level domain (.com, .org, .net...). There are 13 logical root server clusters worldwide.
- **TLD server:** Knows which authoritative server is responsible for a specific domain within that TLD.
- **Authoritative server:** Holds the actual DNS records for the domain and returns the final answer.

2.3 Common DNS Record Types

Record	Purpose	Example
A	Maps a hostname to an IPv4 address	example.com → 93.184.216.34
AAAA	Maps a hostname to an IPv6 address	example.com → 2606:2800:220:1::
CNAME	Alias -- points one name to another name	www.example.com → example.com

Record	Purpose	Example
MX	Specifies mail servers for the domain	example.com → mail.example.com (priority 10)
TXT	Arbitrary text -- often used for verification / SPF / DKIM	"v=spf1 include:_spf.google.com ~all"
NS	Delegates a domain/subdomain to name servers	example.com → ns1.examplehost.com
SOA	Start of Authority -- admin info, zone serial number, TTLs	primary NS, admin email, refresh interval
PTR	Reverse DNS -- maps an IP address back to a hostname	34.216.184.93.in-addr.arpa → example.com

2.4 Caching and TTL

Every DNS record has a Time To Live (TTL), measured in seconds, that tells resolvers how long they may cache the answer before asking again. Caching happens at multiple levels -- the browser, the OS, and the recursive resolver -- which is why DNS changes (e.g. after migrating a server) can take time to propagate: existing cached records must expire first.

2.5 Advanced Concepts (Pro Level)

DNSSEC (DNS Security Extensions)

Plain DNS responses are not authenticated, which makes spoofing possible. DNSSEC adds cryptographic signatures to DNS records so resolvers can verify that a response genuinely came from the authoritative source and was not tampered with in transit, using a chain of trust rooted at the DNS root zone.

Zone transfers (AXFR/IXFR)

Secondary/slave name servers replicate zone data from a primary/master server using zone transfers. Misconfigured servers that allow AXFR from any IP can leak a full map of a domain's internal infrastructure -- a common security misconfiguration.

DNS-based load balancing and failover

Returning multiple A records, or using very low TTLs together with health checks, lets DNS providers direct users to the nearest or healthiest server -- a simple form of load balancing and failover used by CDNs and large-scale services.

Common DNS attacks

- **Cache poisoning / spoofing:** An attacker injects a forged DNS response to redirect users to a malicious server. Mitigated by DNSSEC and randomized query IDs/source ports.
- **DNS tunneling:** Encoding non-DNS data (e.g. exfiltrated data or C2 traffic) inside DNS queries/responses to bypass firewalls that allow DNS traffic by default.
- **DDoS via DNS amplification:** Attackers send small spoofed queries to open resolvers, which reply with much larger responses to the victim's IP, amplifying attack traffic.

Split-horizon / private DNS

Organizations often run internal DNS servers that return different (private) answers for internal clients than the public Internet sees for the same domain name -- useful for keeping internal service names out of public DNS while still using real domain names inside the corporate network.

2.6 Exam Practice: DNS

Q1. Which DNS record type maps a domain name to an IPv4 address?

- a) CNAME
- b) A
- c) MX
- d) NS

Answer: b) A

Q2. Which server type holds the authoritative records for a domain?

- a) Root server
- b) Recursive resolver
- c) TLD server
- d) Authoritative name server

Answer: d) Authoritative name server

Q3. What does a low TTL value on a DNS record achieve?

- a) Faster DNS resolution speed
- b) Records expire from cache sooner, enabling quicker updates
- c) Stronger encryption
- d) Reduced query size

Answer: b) Records expire from cache sooner, enabling quicker updates

Q4. Which technology adds cryptographic authentication to DNS responses?

- a) DNSSEC
- b) DoH
- c) CNAME flattening
- d) TTL

Answer: a) DNSSEC

Q5. Explain the difference between a recursive resolver and an authoritative name server.

Answer: A recursive resolver performs the lookup process on behalf of a client, querying root, TLD, and authoritative servers as needed and caching the result. An authoritative name server is the source of truth for a domain's records and returns the final answer directly.

Q6. Why can DNS changes take time to take effect worldwide even after you update a record?

Answer: Because resolvers and clients cache DNS answers for the duration of the record's TTL. Old, cached values keep being returned until they expire, so propagation is only guaranteed once the previous TTL has fully elapsed everywhere.

Q7. What is DNS amplification, and why is it dangerous?

Answer: It is a DDoS technique where an attacker sends small, spoofed DNS queries to open resolvers using the victim's IP as the source; the resolvers send much larger responses to the victim, amplifying the attacker's traffic and overwhelming the target.

Chapter 3: SSH (Secure Shell)

3.1 What Is SSH and Why Use It?

SSH (Secure Shell) is a cryptographic network protocol used to securely operate network services over an unsecured network -- most commonly to log into a remote machine's command line, transfer files, and tunnel other traffic. It replaced older, insecure protocols like Telnet and rlogin, which sent credentials and data in plain text.

3.2 How an SSH Connection Is Established

SSH runs over TCP, by default on port 22. Setting up a session involves negotiating encryption, verifying the server's identity, and authenticating the client.

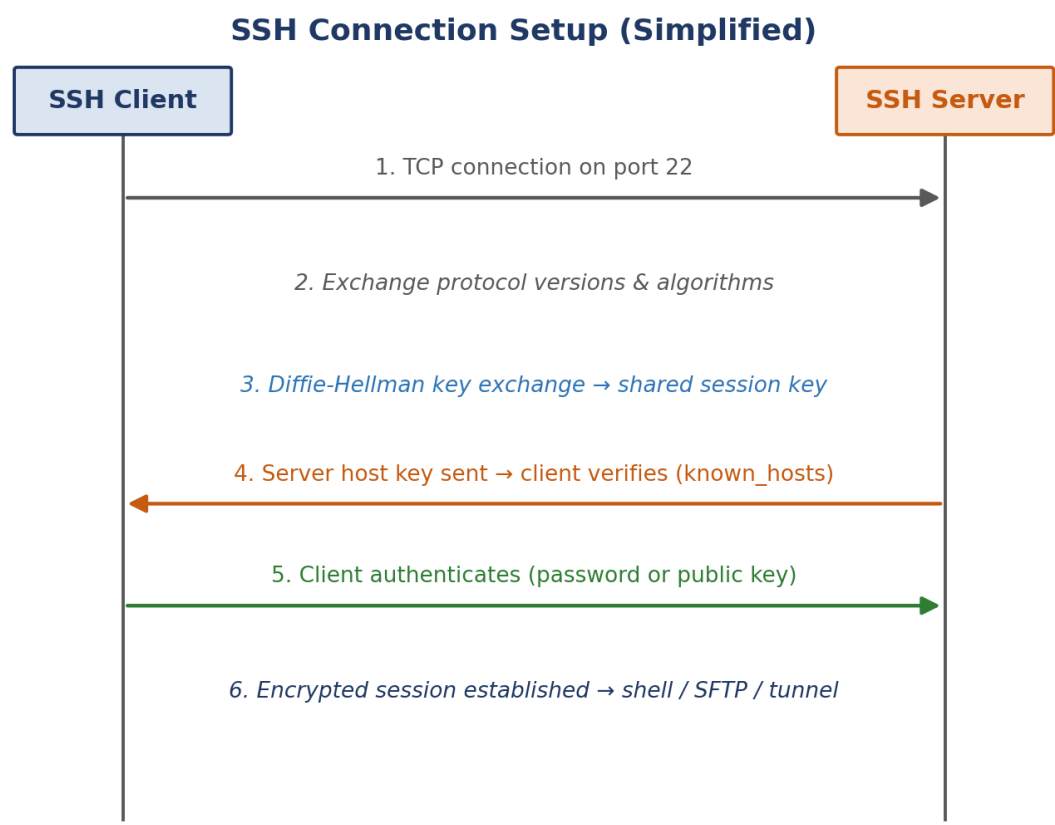


Figure 3.1 — Simplified SSH connection and authentication flow

- **Key exchange:** Client and server use an algorithm such as Diffie-Hellman to agree on a shared session key without ever transmitting it directly -- this key encrypts all further traffic.
- **Host key verification:** The server proves its identity with a host key. The client checks it against its local known_hosts file; a mismatch triggers a security warning (this is what protects against man-in-the-middle attacks after the first connection).
- **Authentication:** The client proves who it is, typically via a password or a public/private key pair.

3.3 Authentication Methods

Method	How it works	Security notes
Password	User types a password, sent over the encrypted channel	Simple but vulnerable to brute-force; often disabled on production servers

Method	How it works	Security notes
Public key	Client proves possession of a private key matching a public key already on the server	Much stronger; the private key never leaves the client
Keyboard-interactive / 2FA	Combines a password/key with an OTP or hardware token	Adds an extra layer, common in hardened environments

Public key authentication in practice

- Generate a key pair: `ssh-keygen -t ed25519`
- Copy the public key to the server: `ssh-copy-id user@server` (appends it to `~/.ssh/authorized_keys`)
- Connect: `ssh user@server` -- the server challenges the client to prove it holds the matching private key, without the key itself ever crossing the network

3.4 The SSH Config File and Common Commands

The `~/.ssh/config` file lets you define per-host shortcuts and defaults instead of typing long commands every time:

- Host myserver → defines an alias
- HostName 203.0.113.10 → the real address
- User devops → default username
- Port 2222 → custom SSH port
- IdentityFile `~/.ssh/id_ed25519` → which private key to use

With this configured, "ssh myserver" connects using all of the above automatically.

3.5 Advanced Concepts (Pro Level)

SSH tunneling / port forwarding

Type	Command pattern	Use case
Local forwarding	<code>ssh -L 8080:internal-host:80 user@server</code>	Access a remote-only service (e.g. an internal dashboard) as if it were local
Remote forwarding	<code>ssh -R 9000:localhost:3000 user@server</code>	Expose a local service to the remote server (e.g. a demo running on your laptop)
Dynamic forwarding (SOCKS proxy)	<code>ssh -D 1080 user@server</code>	Route arbitrary traffic through the server, like a lightweight VPN

SSH agent forwarding

`ssh-agent` caches your decrypted private key in memory so you don't retype your passphrase for every connection. Agent forwarding (`ssh -A`) lets a remote server use your local agent to authenticate onward to a third server -- convenient for jump hosts, but risky: a compromised intermediate server could potentially misuse the forwarded agent, so it should be used carefully and only on trusted hosts.

Jump hosts / bastion hosts

`ssh -J bastion-host target-host` (or a `ProxyJump` entry in the config file) lets you hop through an intermediary server to reach a target that isn't directly reachable from the Internet -- a standard pattern for securing access to private cloud infrastructure.

Hardening SSH servers

- **Disable password authentication:** `PasswordAuthentication no` in `sshd_config`, forcing key-based auth only.

- **Disable root login:** PermitRootLogin no -- log in as a normal user and use sudo instead.
- **Change the default port:** Reduces noise from automated scanners (not a substitute for real security, but cuts log spam).
- **Use fail2ban or similar:** Automatically blocks IPs after repeated failed login attempts.
- **Restrict by IP / firewall:** Only allow SSH from known office/VPN IP ranges where possible.
- **Keep keys protected:** Always use a passphrase on private keys and store them with restrictive file permissions (chmod 600).

3.6 Exam Practice: SSH

Q1. What is the default TCP port for SSH?

- a) 21
- b) 22
- c) 23
- d) 443

Answer: b) 22

Q2. Which SSH authentication method is generally considered most secure?

- a) Password only
- b) Public key authentication
- c) Telnet-style plaintext login
- d) IP whitelisting alone

Answer: b) Public key authentication

Q3. Which SSH option would you use to route your browser traffic through a remote server as a SOCKS proxy?

- a) -L
- b) -R
- c) -D
- d) -J

Answer: c) -D (dynamic forwarding)

Q4. What file does an SSH client check to detect a possible man-in-the-middle attack on first/repeat connection?

- a) authorized_keys
- b) known_hosts
- c) sshd_config
- d) id_rsa

Answer: b) known_hosts

Q5. Explain the difference between local and remote SSH port forwarding.

Answer: Local forwarding (-L) makes a remote service accessible on your local machine by forwarding a local port to a port on/behind the remote server. Remote forwarding (-R) does the opposite -- it exposes a service running on your local machine to the remote server by forwarding a port on the remote server back to your machine.

Q6. Why is disabling password authentication and root login considered a best practice for production servers?

Answer: Passwords are vulnerable to brute-force and credential-stuffing attacks, while key-based authentication is far harder to guess or crack. Disabling direct root login forces attackers (and admins) to authenticate as a named user and then elevate privileges via sudo, which improves auditability and reduces the blast radius of a single compromised credential.

Q7. What risk does SSH agent forwarding introduce, and when is it useful?

Answer: It is useful for authenticating onward through jump/bastion hosts without copying private keys to intermediate servers. The risk is that if an intermediate server is compromised while your agent is forwarded, an attacker could potentially use your forwarded agent to authenticate to other systems as you, so it should only be enabled for trusted hosts.

Chapter 4: SSL/TLS

4.1 What Is SSL/TLS?

SSL (Secure Sockets Layer) and its modern successor TLS (Transport Layer Security) are cryptographic protocols that provide encryption, integrity, and authentication for data sent over a network. "HTTPS" is simply HTTP running inside a TLS-encrypted connection. SSL is now considered obsolete and insecure; all modern traffic uses TLS (commonly still called "SSL" out of habit), with TLS 1.2 and TLS 1.3 in active use today.

TLS provides three core guarantees:

- **Confidentiality:** Data is encrypted so eavesdroppers cannot read it.
- **Integrity:** Data cannot be altered in transit without detection.
- **Authentication:** The client can verify it is really talking to the server it intended to (via certificates).

4.2 Symmetric vs Asymmetric Encryption

	Symmetric Encryption	Asymmetric Encryption
Keys used	One shared secret key for both encryption and decryption	A public key (shared) and a private key (secret), mathematically linked
Speed	Fast -- used for the bulk of the data	Slow -- used only for the initial handshake
Examples	AES, ChaCha20	RSA, ECDSA, Diffie-Hellman
Role in TLS	Encrypts the actual application data after the handshake	Used to authenticate the server and safely establish the shared symmetric key

TLS is a hybrid system: it uses slow asymmetric cryptography just long enough to safely agree on a fast symmetric key, then switches to symmetric encryption for the rest of the session -- getting both security and performance.

4.3 Certificates and PKI (Public Key Infrastructure)

A TLS certificate binds a public key to an identity (e.g. a domain name) and is digitally signed by a Certificate Authority (CA) that browsers and operating systems already trust. This creates a chain of trust from a well-known root CA down to the certificate your server presents.

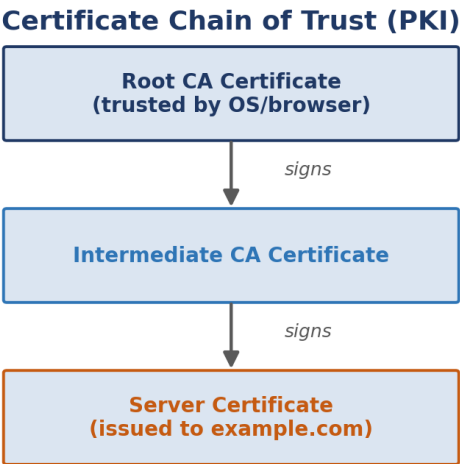
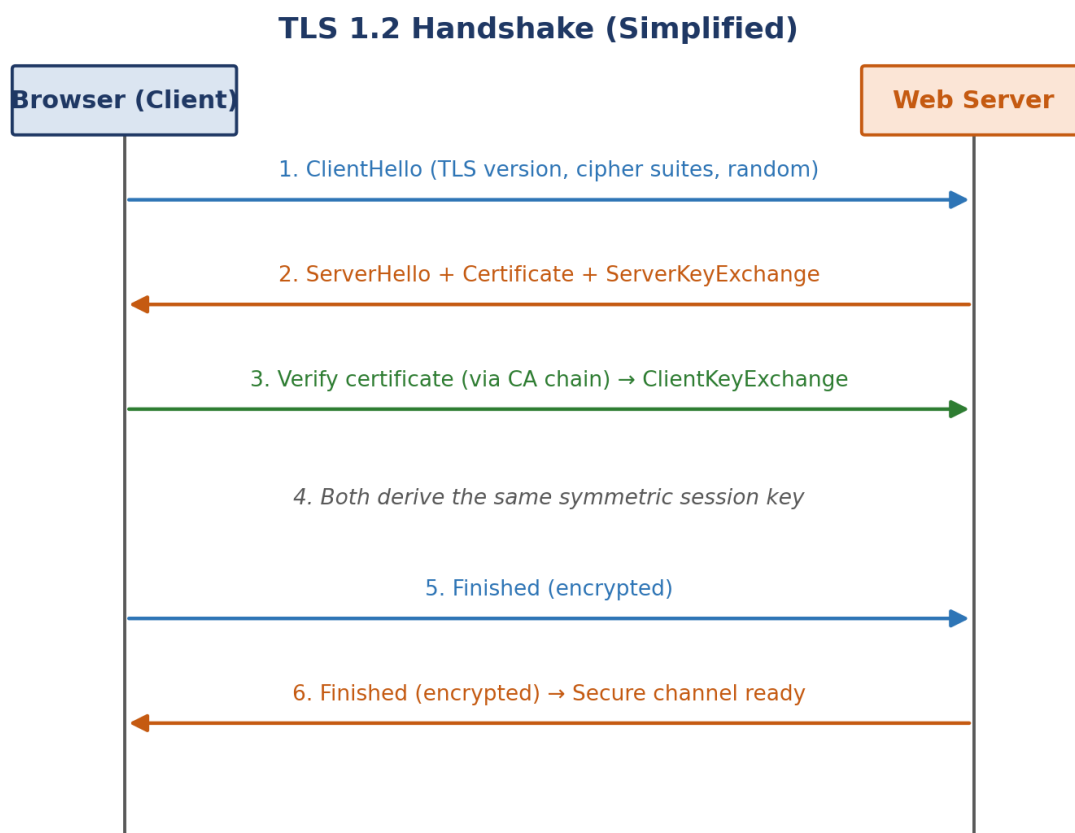


Figure 4.1 — Certificate chain of trust (PKI)

- **Root CA:** A small set of highly trusted authorities (e.g. DigiCert, Let's Encrypt's ISRG Root) pre-installed as trust anchors in operating systems and browsers.
- **Intermediate CA:** Signed by a root CA, used to sign end-entity certificates -- this keeps the root's private key offline and safer.
- **Server (leaf) certificate:** Issued to a specific domain, contains the domain's public key, validity dates, and the issuer's signature.
- **Certificate validation levels:** DV (Domain Validation, proves domain control only), OV (Organization Validation, verifies the company), EV (Extended Validation, the most rigorous identity checks).

4.4 The TLS Handshake

The handshake is where the client and server authenticate each other and agree on a shared symmetric key before any application data is sent.



TLS 1.3 reduces this to a single round trip (1-RTT), improving speed and security.

Figure 4.2 — TLS 1.2 handshake (simplified)

- **ClientHello:** Client proposes a TLS version, supported cipher suites, and a random value.
- **ServerHello + Certificate:** Server picks a cipher suite, sends its certificate (and any intermediates), and its own random value.
- **Key exchange:** Using (EC)DHE or RSA, both sides derive the same symmetric session key without ever sending it directly over the network.
- **Finished:** Both sides send an encrypted "Finished" message; if both can decrypt each other's correctly, the handshake succeeded and the secure channel is ready.

4.5 Advanced Concepts (Pro Level)

TLS 1.3 improvements

- **1-RTT handshake:** TLS 1.3 cuts the handshake down to a single round trip (and offers 0-RTT resumption for repeat connections), reducing latency compared to TLS 1.2's two round trips.
- **Removed weak algorithms:** Legacy, insecure ciphers (RC4, static RSA key exchange, SHA-1, etc.) were dropped entirely.
- **Mandatory forward secrecy:** Every handshake now uses ephemeral (EC)DHE key exchange.

Perfect Forward Secrecy (PFS)

PFS means each session uses a unique, ephemeral key that is discarded afterward. Even if an attacker later steals the server's long-term private key, they cannot decrypt previously captured traffic, because that traffic's session key was never derived from (or recoverable from) the long-term key. This is achieved using ephemeral Diffie-Hellman (DHE/ECDHE) key exchange.

Cipher suites

A cipher suite is a named combination of algorithms used for a TLS session, e.g.

TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384: ECDHE for key exchange, RSA for authentication, AES-256-GCM for symmetric encryption (with built-in integrity), and SHA-384 for the handshake's hash function.

Notable historical vulnerabilities

Vulnerability	Issue
Heartbleed (2014)	A buffer over-read bug in OpenSSL's heartbeat extension leaked server memory (potentially including private keys and session data)
POODLE (2014)	Exploited a padding oracle in SSL 3.0, allowing attackers to decrypt parts of encrypted traffic; led to SSL 3.0 being deprecated
BEAST (2011)	Exploited a weakness in TLS 1.0's CBC cipher chaining, mitigated by moving to TLS 1.1+/1.2 and different cipher handling

Mutual TLS (mTLS)

In standard TLS, only the server presents a certificate. In mutual TLS, the client also presents a certificate, and the server validates it -- used heavily in service-to-service (microservice) communication and zero-trust architectures where both ends must prove their identity.

4.6 Exam Practice: SSL/TLS

Q1. Which type of encryption is used for the bulk of data transfer after a TLS handshake completes?

- a) Asymmetric encryption
- b) Symmetric encryption
- c) Hashing only
- d) No encryption is used

Answer: b) Symmetric encryption

Q2. What is the primary purpose of a Certificate Authority (CA)?

- a) To encrypt application data
- b) To generate symmetric session keys
- c) To vouch for the identity behind a public key by signing certificates
- d) To route TLS traffic between servers

Answer: c) To vouch for the identity behind a public key by signing certificates

Q3. How many round trips does a full TLS 1.3 handshake typically require?

- a) 0
- b) 1
- c) 2
- d) 3

Answer: b) 1 (1-RTT, with 0-RTT possible on resumption)

Q4. What property does Perfect Forward Secrecy guarantee?

- a) Certificates never expire
- b) Past session traffic stays safe even if the server's long-term key is later compromised
- c) Passwords are hashed
- d) DNS records are authenticated

Answer: b) Past session traffic stays safe even if the server's long-term key is later compromised

Q5. Explain why TLS uses both asymmetric and symmetric encryption instead of just one.

Answer: Asymmetric encryption is secure for exchanging a key without prior shared secrets but is computationally slow. Symmetric encryption is fast but requires both sides to already share a secret key. TLS uses asymmetric cryptography briefly during the handshake to safely establish a shared symmetric key, then uses that fast symmetric key to encrypt the actual application data.

Q6. What is the difference between a root CA and an intermediate CA?

Answer: A root CA is a highly trusted authority whose certificate is pre-installed as a trust anchor in operating systems/browsers; its private key is kept offline for security. An intermediate CA is signed by the root CA and does the day-to-day work of signing end-entity (server) certificates, isolating the root's private key from routine exposure.

Q7. What was the core flaw exploited by Heartbleed, and why was it serious?

Answer: Heartbleed was a buffer over-read vulnerability in OpenSSL's heartbeat extension: a malformed heartbeat request could trick a server into returning up to 64KB of adjacent memory. It was serious because that memory could contain private keys, session tokens, passwords, or other sensitive data, and the bug could be exploited repeatedly without leaving obvious traces.